

Incident Response — Detection

Lab: Incident Response — Detection with Wazuh **Role:** Junior SOC Analyst **Companion to:** 14_Incident_Response_Detection_Learner_Lab_Document **Curriculum position:** Day 14 of the cybersecurity bootcamp

How to Use This Document

This Hints Companion sits alongside the Lab Document. The Lab Document tells you *what* to do; this companion offers *help* when you get stuck. Use it after you have attempted a task — not before.

Each task section uses five callout types:

💡 **Hint** — A nudge that points you toward a tool, flag, or concept without giving the full answer. Use this when you have an idea but cannot find the right syntax.

🔍 **What to look for** — The specific lines or fields in a command's output (or in a Wazuh panel) that actually matter for the task.

⚠️ **Common mistake** — A pitfall that produces silently-wrong output: an empty file, the wrong agent filter, a default flag missed. Read these before you submit.

✅ **You're done when** — A self-check you can run to confirm the task is complete and the evidence is acceptable.

📄 **Sample output** — A trimmed example of what correct output looks like. Real values; the same shape your output should take.

Task 1 — Verify the Kali workstation and reach the target

💡 **Hint** — Three pieces of information go into the file: who you are (hostname), where you are (IP/interface), and that you can reach the target. There are short commands for each.

💡 **Hint** — Capture everything into one file by chaining commands with shell redirection. Using `>` overwrites the file each time; `>>` appends. You want to append after the first command so you do not lose what came before.

💡 **Hint** — If `ping` hangs, send a fixed number of packets so the command finishes on its own.

🔍 **What to look for** — In your IP output, find the line beginning `inet` for the lab-facing interface (usually `eth0` or `ens33`). In the ping output, find lines beginning `64 bytes from` and the summary line showing `0% packet loss`.

⚠️ **Common mistake** — Forgetting to redirect to a file. Watching output scroll past in the terminal is not evidence. Open the file afterward with `cat` and confirm the content is really there.

⚠️ **Common mistake** — Capturing the wrong interface's IP (the loopback `127.0.0.1` instead of the lab interface). Loopback never proves you can reach DC10.

📄 Sample output —

```
kali@kali:~$ hostname
kali

eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 10.10.10.50 netmask 255.255.255.0 broadcast 10.10.10.255

PING 10.10.10.10 (10.10.10.10) 56(84) bytes of data.
64 bytes from 10.10.10.10: icmp_seq=1 ttl=128 time=1.42 ms
64 bytes from 10.10.10.10: icmp_seq=2 ttl=128 time=1.18 ms
--- 10.10.10.10 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss
```

✓ **You're done when** — `cat task01_kali_identity.txt` shows your hostname, the `inet` line for your lab interface, and at least two `64 bytes from 10.10.10.10` lines.

Task 2 — Prepare the password list

💡 **Hint** — The source file lives somewhere under `/usr/share/wordlists/`. Use `find` or `locate` to track it down if the exact path differs from the lab brief.

💡 **Hint** — Copy first, then edit. Editors like `nano` and `vi` are fine; so is `sed` if you prefer the command line.

💡 **Hint** — You want `Pa$$w0rd` to be a known line — not line 1 (too easy, hydra finds it instantly) and not line 500 (the attack takes forever). Somewhere in the middle, around line 200–300, is more realistic.

🔍 **What to look for** — After saving, `grep -n 'Pa$$w0rd' passlist.txt` should return one and only one line. The number before the colon is the line position you record in your report.

⚠ **Common mistake** — Editing the original under `/usr/share/wordlists/`. Other learners share this VM image. Always work on a copy in your lab folder.

⚠ **Common mistake** — Adding the password as `Pa$$w0rd` through an interactive editor and inserting a trailing space or tab by accident. Hydra compares the password byte-for-byte; a trailing space means it will never match.

⚠ **Common mistake** — Forgetting the dollar signs are literal characters. If you are using a tool that does shell expansion, you may end up writing `Paw0rd` (empty variables) into the file. Single-quote the string or use an editor.

📄 Sample output —

```
kali@kali:~/ir_detection_lab$ wc -l passlist.txt
501 passlist.txt
kali@kali:~/ir_detection_lab$ grep -n 'Pa$$w0rd' passlist.txt
247:Pa$$w0rd
```

✓ **You're done when** — The file has at least 100 lines, `Pa$$w0rd` appears exactly once, and you know which line number it sits on.

Task 3 — Authenticate to the Wazuh dashboard and filter to DC10

💡 **Hint** — Firefox will throw a certificate warning the first time you hit the Wazuh URL because the lab uses a self-signed cert. “Advanced” → “Accept the Risk and Continue” is the path.

💡 **Hint** — In the Wazuh menu (hamburger icon, top-left in most builds), look for **Security events**, or under **Modules** look for **Threat Hunting**. The exact label depends on Wazuh version.

💡 **Hint** — To filter by agent, type in the search bar: `agent.name: DC10` (or use the agent ID instead of the name). Press Enter or click **Update / Refresh**.

🔍 **What to look for** — A filter “chip” or pill appearing under the search bar showing the active filter. The events table should now show only DC10 events, and the agent column should be uniform.

⚠️ **Common mistake** — Confusing the Wazuh password `Pa??w0rd` (two question marks) with the Windows password `Pa$$w0rd` (two dollar signs). They are different on purpose. If login fails, check which one you typed.

⚠️ **Common mistake** — Leaving the time range at “Today” when DC10 events you need are older or vice-versa. Use “Last 24 hours” while doing the lab.

✅ **You’re done when** — The screenshot clearly shows the Security events header, the active `agent.name: DC10` filter, the time picker, and at least one event row.

Task 4 — Run a password-guessing attack against RDP on DC10

💡 **Hint** — Hydra’s manpage (`man hydra`) and `hydra -h` are your friends. The flags you need: one for the username, one for the password file, one for the protocol/service module, and the target host.

💡 **Hint** — The service module for RDP in hydra is literally `rdp`. The target is positional and comes last.

💡 **Hint** — To save hydra’s full output to a file while still seeing it on screen, pipe through `tee`.

💡 **Hint** — A single fixed username uses `-l` (lowercase L). A password list file uses `-P` (uppercase P). Get these backwards and hydra reads `Administrator` as a one-line wordlist.

🔍 **What to look for** — When hydra finds a valid credential it prints a line that contains `host:`, `login:`, and `password:` on the same line. That is the success indicator.

⚠️ **Common mistake** — Running the attack with `-l` and `-p` (lowercase, single password) instead of `-l` and `-P` (uppercase, password file). Lowercase `-p` uses the next argument as a literal password and ignores your wordlist entirely.

⚠️ **Common mistake** — Running hydra against the wrong host. Double-check the target IP. Generating brute-force traffic at any host other than DC10 violates lab scope.

⚠️ **Common mistake** — Skipping output capture. If hydra’s success line scrolls off the terminal and you never wrote it to a file, you have no evidence.

Sample output —

```
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations...

[DATA] attacking rdp://10.10.10.10:3389/
[STATUS] 64.00 tries/min, 64 tries in 00:01h, 437 to do in 00:07h
[3389] [rdp] host: 10.10.10.10  login: Administrator  password: Pa$$w0rd
1 of 1 target successfully completed, 1 valid password found
```

✓ **You're done when** — `grep 'password:' task04_hydra_output.txt` returns a line containing `login: Administrator` and `password: Pa$$w0rd`.

Task 5 — Identify the Wazuh alerts for the brute-force activity

💡 **Hint** — Refresh the Security events page (or click **Update**) right after hydra finishes. Wazuh ingests events in near real-time, but you may need to widen the time picker to “Last 15 minutes”.

💡 **Hint** — To filter to a specific rule, click on the Rule ID in the events table, or add `rule.id: 60122` to the search bar.

💡 **Hint** — Click any single event row to expand its detail panel on the right. Within that panel, look for the **rule** object and the **mitre** object.

🔍 **What to look for** — In the expanded event JSON, you want these fields: `rule.id`, `rule.level`, `rule.description`, `rule.mitre.tactic`, `rule.mitre.technique`, `rule.mitre.id`.

⚠ **Common mistake** — Capturing a screenshot of the events list (the row level) rather than the expanded detail panel. The list does not show MITRE fields; the detail panel does.

⚠ **Common mistake** — Filtering by Rule ID and forgetting to re-add the agent filter. You may end up looking at 60122 events from a different host. Always confirm `agent.name` matches DC10.

⚠ **Common mistake** — Recording only one 60122 alert. Hydra produces dozens of failed-logon events per minute during the run. Note the count, not just one example.

✓ **You're done when** — You have one screenshot each for 60122 and 92652, both showing the expanded detail panel including the rule level, description, and a non-empty MITRE technique field.

Task 6 — Examine the MITRE ATT&CK mapping for Rule 92652

💡 **Hint** — In the alert detail panel, the technique value is usually a clickable link. Click it to jump to the technique page either inside Wazuh's intelligence module or on `attack.mitre.org`.

💡 **Hint** — Hydra performed a brute-force attack over RDP. The MITRE family for that is **T1110 — Brute Force**, with sub-techniques for password guessing, password cracking, password spraying, and credential stuffing. Your job is to decide which sub-technique most accurately describes what you did.

💡 **Hint** — Wazuh's default mapping for 92652 may say something more generic (such as just “Brute Force” without a specific sub-technique). That is the point of this task: the analyst sometimes has to be more specific than the default.

🔍 **What to look for** — On the technique page, the technique ID (e.g., `T1110.001`), the technique name, and the parent tactic (**Credential Access**).

⚠ **Common mistake** — Accepting Wazuh's mapping uncritically. If the default says "Valid Accounts" (T1078) but you ran a password-guessing attack, the more accurate technique is "Brute Force" (T1110), with "Password Guessing" (T1110.001) as the specific sub-technique. Note the discrepancy in your report.

⚠ **Common mistake** — Writing a vague analyst note ("the mapping seems okay"). Be specific: which technique, which sub-technique, why this one fits better.

✅ **You're done when** — Your screenshot shows the MITRE technique page and your report has a one-paragraph analyst note that names the technique ID you would prefer and explains why.

Task 7 — Attempt to mount the DC10 administrative share with invalid credentials

💡 **Hint** — The mount type for Windows SMB shares is `cifs`. You need a local empty directory as the mount point — create one under `/mnt/` with `mkdir` first.

💡 **Hint** — Pass credentials via the `-o` options string: `username=...`, `password=...`, and (for a workgroup-style account) `domain=...` if needed.

💡 **Hint** — `mount` writes errors to stderr. To capture them, redirect both stdout and stderr: `2>&1` or `&>`.

💡 **Hint** — You will likely need `sudo` for the mount command.

🔍 **What to look for** — On failure, expect a message like `mount error(13): Permission denied`, or `mount error: Authentication failure`. The exact wording depends on cifs-utils version.

⚠ **Common mistake** — Writing the password literally as `Paw0rd` *< /code>* *> onthecommandlinewithoutquoting. Theshellwillexpand < code>* to the current shell PID. Always single-quote the password or wrap the whole options string in single quotes.

⚠ **Common mistake** — Forgetting to record the exact local time of the attempt. You need it to correlate with the Wazuh timestamp in Task 9.

📄 Sample output —

```
kali@kali:~/ir_detection_lab$ sudo mount -t cifs //10.10.10.10/C$/mnt/dc10 -o 'username=Jaime,password=Pa$$w0rd' 2>&1 | t
mount error(13): Permission denied
Refer to the mount.cifs(8) manual page (e.g. man mount.cifs) for details
```

✅ **You're done when** — Your output file contains a permission-denied or authentication-failure error, and `mount | grep cifs` shows nothing mounted.

Task 8 — Mount the DC10 administrative share with valid credentials

💡 **Hint** — The command is identical to Task 7 except for the username. Use `Administrator` instead of `Jaime`.

💡 **Hint** — Once mounted, `ls /mnt/dc10` will list the share's root. Capture this in your output file.

💡 **Hint** — Always unmount cleanly with `sudo umount /mnt/dc10` when you are finished. Leaving stale mounts behind can cause issues for the next lab.

🔍 **What to look for** — On success, the mount command returns silently (no error). The `ls` output shows typical Windows top-level directories: `Users`, `Windows`, `Program Files`, `Program Files (x86)`, `ProgramData`, etc.

⚠️ **Common mistake** — Listing the contents but not capturing them to the output file. Pipe to `tee` or redirect both the mount command and the `ls`.

⚠️ **Common mistake** — Forgetting to unmount before moving on. Run `mount | grep cifs` at the end to confirm cleanup.

⚠️ **Common mistake** — Quoting the password incorrectly again. The same shell-expansion pitfall from Task 7 applies here.

📄 Sample output —

```
kali@kali:~/ir_detection_lab$ sudo mount -t cifs //10.10.10.10/C$ /mnt/dc10 -o 'username=Administrator,password=Pa$$w0rd'
kali@kali:~/ir_detection_lab$ ls /mnt/dc10
'$Recycle.Bin'  PerfLogs  'Program Files'  'Program Files (x86)'  ProgramData  Users  Windows
kali@kali:~/ir_detection_lab$ sudo umount /mnt/dc10
```

✅ **You're done when** — Your output file shows the successful directory listing, and a final `mount | grep cifs` returns nothing.

Task 9 — Correlate the mount attempts to Wazuh alerts

💡 **Hint** — Refresh Security events for DC10 and narrow the time window to the period during which you ran Tasks 7 and 8 (e.g., "Last 15 minutes").

💡 **Hint** — The failed mount fires Rule ID 60122 (same rule family as failed RDP logons — it is a Windows audit failure event). The successful mount fires Rule ID 60106 (Windows successful logon).

💡 **Hint** — To zero in, filter the search bar with `rule.id: 60106 OR rule.id: 60122`. Then expand individual events to confirm the source IP is your Kali host and the username matches what you used.

🔍 **What to look for** — In each alert's detail, the `data.win.eventdata.targetUserName` field should show `Jaime` (for the failed mount) and `Administrator` (for the successful one). The `data.win.eventdata.ipAddress` should match your Kali IP. The MITRE technique reference will likely include `T1078` (Valid Accounts).

⚠️ **Common mistake** — Mistaking a 60122 alert from the earlier hydra run for a Task 7 alert. Use timestamps. The mount events happened minutes after hydra finished.

⚠️ **Common mistake** — Recording the MITRE technique as just "T1078" without noting that this is a different technique from the brute-force-related ones in Task 5. The analyst's job is to notice that different attacker actions produce different mappings.

✓ **You're done when** — You have one screenshot per Rule ID, each showing the alert details with the correct username (`Jaime` for 60122, `Administrator` for 60106) and the Kali IP as the source.

Task 10 — Clear the Windows Security event log on DC10

💡 **Hint** — Connect to DC10 by clicking into the DC10 VM console window directly inside VMware on your Windows 11 host. This avoids piling on more RDP alerts in Wazuh.

💡 **Hint** — Press the Windows key inside DC10 and type `eventvwr` to launch Event Viewer. Expand **Windows Logs** in the left pane.

💡 **Hint** — Right-click **Security** → **Clear Log...** → **Clear** (the “without saving” option is fine for this lab).

🔍 **What to look for** — Immediately after clearing, the Security log refreshes and shows a single new event: **Event ID 1102 — The audit log was cleared**. That single event is the only entry the local log now has.

⚠️ **Common mistake** — Clearing the System or Application log by mistake. Only the Security log is in scope. Read the left-pane selection carefully before clicking **Clear Log**.

⚠️ **Common mistake** — Not recording the local DC10 time when you cleared the log. You need it for the report timeline and for matching to the Wazuh-side timestamp in Task 11.

⚠️ **Common mistake** — Capturing the screenshot before the clear, not after. The evidence shows the post-clear state.

✓ **You're done when** — Your DC10 screenshot shows Event Viewer with the Security log selected, the log empty or containing only Event 1102, and the timestamp of that 1102 event visible.

Task 11 — Confirm Wazuh detected the log-clearing event

💡 **Hint** — Back in Wazuh, refresh Security events for DC10. The log-clear alert should appear within seconds.

💡 **Hint** — Filter by `rule.id: 63103`. If you do not see a hit at exactly that ID, search by description instead: `rule.description: audit log was cleared` or `rule.description: log cleared`. Wazuh rule IDs occasionally vary by ruleset version.

💡 **Hint** — In the alert detail, you should see the source event ID (1102) inside the Windows event data, confirming Wazuh ingested the underlying Windows event before the local log was wiped.

🔍 **What to look for** — A high-severity alert (typically Level 12 or higher), a description mentioning audit log clearing, the agent **DC10**, and a MITRE technique reference to **T1070** (Indicator Removal), often the sub-technique **T1070.001** (Clear Windows Event Logs).

⚠️ **Common mistake** — Assuming Wazuh failed because the exact rule ID 63103 does not appear. Different Wazuh ruleset versions number this rule differently. Search by description, capture whatever Rule ID Wazuh actually shows, and record that ID in your report.

⚠️ **Common mistake** — Not making the connection in writing: the local Windows Security log has only Event 1102 in it now, but Wazuh has the full history of pre-clear events because the agent forwarded them in real time. State this explicitly in your report.

✓ **You're done when** — Your Wazuh screenshot shows the log-clear alert detail panel including the Rule ID, the DC10 agent, the description, the level, and the MITRE technique containing T1070.

Task 12 — Write the incident detection report

💡 **Hint** — Open `report_template.md` in your preferred editor on Kali. Work top to bottom; do not skip sections.

💡 **Hint** — The timeline is the most valuable single artifact in this report. Order rows by timestamp. Tier 2 analysts read the timeline first.

💡 **Hint** — For your “Why centralized logging matters” paragraph, anchor it to a specific observation: “If Wazuh had not forwarded events before the Security log was cleared at HH:MM, we would have lost evidence of the X failed RDP attempts and the successful logon at HH:MM.”

🔍 **What to look for** — Each section in the template has placeholders in `italic em-dash` style. After your edit, none of these should remain.

⚠ **Common mistake** — Leaving the placeholder text (`value`, `Your name`) in place. Reviewers downgrade reports that have unfilled placeholders.

⚠ **Common mistake** — Pasting raw screenshots into the report. The report is the narrative; the screenshots are separate evidence files. Reference them by filename in section 15.

⚠ **Common mistake** — Writing the report from memory instead of from your captured files. Open `task04_hydra_output.txt` and the screenshots while you write, and quote precise values.

✓ **You're done when** — Every section is filled, the timeline has at least 5 entries with real timestamps, the MITRE-mapping analyst note (section 7) is at least three sentences, and the evidence list (section 15) names every file in your zip.

Wazuh Rule ID cheat sheet

Rule ID	Description	Typical MITRE mapping	Severity
60106	Windows successful logon	T1078 — Valid Accounts	Low-Medium
60122	Windows failed logon (audit failure)	T1110 — Brute Force (when repeated)	Medium
92652	Multiple Windows logon failures followed by a success — possible password guessing	T1110 — Brute Force	High
63103	Windows audit log was cleared (maps to Windows Event 1102)	T1070.001 — Clear Windows Event Logs	High-Critical
60103	Windows logon special privileges assigned	T1078 — Valid Accounts	Medium

Rule IDs and exact mappings vary by Wazuh ruleset version. If your environment's IDs differ slightly, record what you actually observe rather than what this table predicts.

Command quick reference

Command	Purpose
<code>hostname</code>	Print the Kali workstation's hostname
<code>ip a</code>	List network interfaces and IP addresses
<code>ping -c 4 <host></code>	Send 4 ICMP echo requests and stop
<code>cp <src> <dst></code>	Copy a wordlist into your lab folder
<code>nano <file></code> / <code>vi <file></code>	Edit a text file interactively
<code>grep -n '<string>' <file></code>	Find a string and print its line number
<code>wc -l <file></code>	Count lines in a file
<code>hydra -l <user> -P <file> rdp://<host></code>	Run a password-guessing attack against RDP
<code>tee <file></code>	Duplicate stdout to both terminal and file
<code>mkdir /mnt/<name></code>	Create a mount-point directory
<code>sudo mount -t cifs //<host>/<share> <mountpoint> -o 'username=...,password=...'</code>	Mount a Windows SMB share
<code>mount grep cifs</code>	Verify which CIFS shares are mounted
<code>sudo umount <mountpoint></code>	Unmount a share cleanly
<code>eventvwr.msc</code>	Open Event Viewer on Windows

Troubleshooting

Ping to DC10 times out The DC10 VM may be powered off, or your Kali and DC10 VMs may be on different VMware virtual networks. Confirm both VMs are powered on in VMware on your Windows 11 host. Check that both VM network adapters are attached to the same VMnet (usually a custom or host-only network). From Kali, confirm `ip a` shows an IP in the same subnet as DC10.

Hydra returns “0 valid passwords found” Either `Pa$$w0rd` is not in your wordlist (re-check Task 2 with `grep -n`), the password file has a trailing space on that line, or you used `-p` (single password literal) instead of `-P` (wordlist file). Re-check your command and the file.

Hydra reports “Connection refused” on every attempt RDP (3389/tcp) is not reachable on DC10. Confirm with a port check or simply with `ping` first. If ping works but RDP does not, the Windows firewall or RDP service is blocking; flag this to your team lead.

Mount returns “Host is down” DNS or the host route is broken between the nested VMs. Use the IP `10.10.10.10` directly instead of the name `DC10`.

Mount returns “Permission denied” with the Administrator credentials You may have a typo in the password — double-check the dollar signs. Or the shell expanded `$$` into a PID number; re-issue the command with single quotes around the whole options string.

Wazuh login fails with the correct-looking password You are probably typing `Pa$$w0rd` (the Windows password) instead of `Pa??w0rd` (the Wazuh password). These differ on purpose. Use the question marks for Wazuh.

Wazuh dashboard shows no events at all The time picker may be set to a window that excludes “now”. Click the time picker and choose **Last 15 minutes**. Then click **Refresh**. Also verify the Wazuh Manager VM is fully booted on your Windows 11 host.

Cannot find Rule 63103 after clearing the log The exact rule ID for “Windows audit log was cleared” varies by ruleset version. Search by description (`*log was cleared*`) or by Windows Event ID (`data.win.system.eventID: 1102`)

instead. Record whichever Rule ID you actually find.

Firefox keeps showing “Connection not secure” The Wazuh dashboard uses a self-signed certificate. Click **Advanced → Accept the Risk and Continue**. This is expected in the lab; do not do this on production systems without verifying the certificate.

Wazuh session keeps timing out The dashboard logs out idle sessions. Just sign back in with `admin` / `Pa??w0rd`. Your filters do not persist across sessions; you will need to re-apply the `agent.name: DC10` filter.

Nested VM is sluggish or freezes The Windows 11 host needs enough free RAM and CPU to run all three nested VMs at once. Close other Windows applications. In VMware, lower the Kali VM’s allocated RAM if needed, but do not drop the Wazuh Manager VM’s RAM below the documented minimum.

Final pre-submission checklist

- ☐ `task01_kali_identity.txt` exists and contains hostname, IP, and successful ping output.
- ☐ `passlist.txt` exists, has ≥ 100 lines, and contains exactly one `Pa$$w0rd` line.
- ☐ `task04_hydra_output.txt` contains a `login: Administrator password: Pa$$w0rd` success line.
- ☐ `task07_mount_invalid.txt` contains a permission-denied or auth-failure message.
- ☐ `task08_mount_valid.txt` shows a successful directory listing of the DC10 C\$ share.
- ☐ All 12 screenshots listed in Section 8 of the Lab Document are present in `screenshots/` and named exactly as specified.
- ☐ Each screenshot shows the required detail (Wazuh detail panel, not just the events list, where applicable).
- ☐ `ir_detection_report.md` is complete, with no `_em-dash placeholder_` text left behind.
- ☐ The report timeline has at least 5 rows with real timestamps.
- ☐ The MITRE-mapping analyst note (report section 7) names a specific technique ID and explains the reasoning.
- ☐ The “Why centralized logging matters” paragraph references your specific Task 11 observations.
- ☐ No stale CIFS mounts remain (`mount | grep cifs` returns nothing).
- ☐ The zip filename matches the convention `ir_detection_lab_<yourname>.zip`.
- ☐ Submission status in the portal reads **Submitted — Pending Review** (or the email fallback was used).